Intra-Trajectory Error Balancing and Inter-Trajectory Feature Point Clustering for Trajectory Compression

Lei Yang, *Member, IEEE*, Xin Cheng, Yuwei Liao, Rui Li, *Member, IEEE*, and Guoqi Xie, *Senior Member, IEEE*

***Abstract*—The widespread use of locatable devices leads to a sharp increase in the storage of trajectory data, and redundant storage of similar trajectories wastes a large amount of storage resources. The state-of-the-art multiple trajectory compression algorithms are developed to strip the partial information of trajectory; however, these algorithms have low compression efficiency because they do not eliminate the redundancy within a single trajectory as much as possible, as well as high time overhead due to matching of reference sub-trajectories. In this study, we propose a new spatio-temporal trajectory compression technique, consisting of *intra-trajectory error balancing* and *inter-trajectory feature point clustering*. Intra-trajectory error balancing is achieved through retaining high score (an aggregated metric) trajectory points (i.e., feature points). Furthermore, inter-trajectory feature point clustering realizes the fusion of similar trajectories and extracts the commonality between trajectories. Experiments are performed on five real trajectory datasets, including two road datasets, one airline dataset, and one walking dataset. Compared with the state-of-the-art methods, our compression technique improves the compression ratio by an average of 24.9% under the same error, and reduces the time overhead by at least an order of magnitude.**

***Index Terms*—Compression algorithm, similar trajectory, spatio-temporal data.**

I. Introduction

A. Background and Motivation

IN RECENT years, trajectory data mining has been widely used in numerous fields, including path recommendation, position prediction, and motion behavior analysis, among others, contributing significantly to the development of position acquisition technology. A variety of technologies currently provide location services, such as Global Positioning System (GPS), Radio Frequency IDentification (RFID), and smartphone sensors. With an increasing number of devices equipped with GPS modules, the amount of trajectory data collected by GPS is also on the rise, which brings a great storage burden to the data center [1], [2], [3], [4], [5]. Movement trajectory data typically records the position of a moving object at fixed or random intervals. For instance, storing nearly a thousand targets in a day would require 2 GB of storage if data were collected at 2-second intervals [6].

Generally, trajectories have a predetermined destination, and if such a trajectory occurs with a certain frequency, they are usually similar. These trajectories are dominant in urban land navigation, such as buses and vans on fixed routes, commuting routes for workers, and so on. Similarly, in unconstrained environments like air and water navigation, moving objects follow long and consistent paths, and different objects may also produce similar trajectories [7]. However, storing these trajectories in their entirety can be cumbersome, as it requires significant storage costs. Fortunately, such data often contains a lot of redundant information, which provides an opportunity for compression. Therefore, it is a critical issue to reduce the storage overhead of trajectory data by exploiting intra-trajectory and inter-trajectory redundancy while ensuring its availability.

The Single Trajectory Compression (STC) algorithm is commonly used to simplify trajectories based on an error metric [8], [9], [10], [11]. Given an error threshold, the trajectory points exceeding the threshold are retained as feature points. This ensures that the maximum error of the simplified trajectory is no greater than the error threshold while minimizing its length, which is called the *Min-Size* problem [12], [13]. However, a single error metric makes it difficult for these algorithms to guarantee information loss in other features, resulting in disproportionate information loss.

The Multiple Trajectory Compression (MTC) algorithm extracts the commonality between similar trajectories to further reduce storage costs [14]. TrajStore is an adaptive trajectory storage system [15] and it reduces redundancy between trajectories by time mapping technique, that is, sharing spatial information among similar trajectories and storing only the time mapping values of the reference trajectory. However, the compression gains brought by the time mapping technique are relatively low. To improve the compression ratio, the Similarity-based MTC (SMTC) algorithm [16] performs optimal storage based on the STC algorithm and the time mapping technology with all compressed trajectories as reference, which improves the compression ratio and also brings high time complexity. REference-based Spatio-Temporal compression (REST) [17], [18] is the latest MTC technology, which compresses trajectories by constructing a large set of reference sub-trajectories. However, REST appears to prioritize commonalities between

Received 23 July 2024; revised 30 May 2025; accepted 9 June 2025. Date of publication 13 June 2025; date of current version 24 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372167 and in part by the Natural Science Foundation of Hunan Province of China under Grant 2024JJ5097. Recommended for acceptance by K. Zheng. *(Lei Yang and Xin Cheng contributed equally to this work.) (Corresponding author: Rui Li.)*

Lei Yang, Xin Cheng, Yuwei Liao, Rui Li, and Guoqi Xie are with the School of Information Science and Engineering, Hunan University, Changsha 410082, China, and also with the Research Institute, Hunan University, Chongqing 401151, China (e-mail: jt_yl@hnu.edu.cn; chengxin@hnu.edu.cn; liaoyuwei23@hnu.edu.cn; rui@hnu.edu.cn; xgqman@hnu.edu.cn).

Digital Object Identifier 10.1109/TKDE.2025.3579434

1041-4347 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

trajectories over redundancy within the trajectories themselves. In actual compression, the time overhead of REST to match the reference sub-trajectory is large, and the construction of the reference sub-trajectories set also has a great influence on the compression effect. The limitations of the abovementioned MTC algorithms are as follows.

1) Time mapping technique [15], [16] involves multiple traversals of the trajectory and may introduce the trajectory dependence problem, which leads to high time overhead (including compression time and recovery time). Taking SMTC [16] as an example, T_{a} is the reference for T_{b}, and T_{b} is the reference for T_{c}. When restoring the T_{c}, it is necessary to restore and traverse the data of T_{a} and T_{b} in advance.
2) Reference-based compression [17], [18] focuses less on redundancy inside trajectories, which limits the overall compression ratio. Additionally, the matching of the reference sub-trajectory set brings high time overhead, and the training and expansion of reference sub-trajectories is also a laborious process.
3) It is difficult to balance the relationship between storage cost and error. For TrajStore [15] and SMTC [16], if the time value obtained by mapping maintains a high accuracy, it means that a better compression effect cannot be achieved; otherwise, the recovered trajectory error becomes larger. For REST [17], [18], a larger error threshold is required to achieve a higher compression ratio.

B. Our Contributions

In this study, we propose a new spatio-temporal trajectory compression technique, consisting of intra-trajectory error balancing and inter-trajectory feature point clustering. The contributions of the proposed technique are summarized as follows.

1) For intra-trajectory error balancing, we propose a Score-based Error Balancing (SEB) algorithm to eliminate redundancy within the trajectory. SEB adopts the idea of divide and conquer to solve the *Min-Error* problem and unbalanced information loss that commonly exists in the STC algorithm.
2) For inter-trajectory feature point clustering, we propose a Cluster-based Feature Point Fusion (CFPF) algorithm by utilizing the cluster distribution characteristics of the feature points in similar trajectories to eliminate redundancy between trajectories. CFPF reduces the granularity of clustering from trajectory segments to feature points and provides a solution to the performance bottleneck problem faced by the current MTC algorithms.
3) Extensive experiments based on five real trajectory datasets demonstrate the effectiveness of our proposed technique. Compared with the state-of-the-art methods, our compression technique improves the compression ratio by an average of 24.9% under the same error, and reduces the time overhead by at least an order of magnitude.

The rest of this paper is organized as follows. Section II reviews related work. Sections III and IV propose the SEB and CFPF algorithms, respectively. Section V verifies the performance of the presented SEB and CFPF algorithms, and Section VI concludes this study.

II. Related Work

In this section, we review related algorithms for trajectory compression. According to whether the redundancy between trajectories is eliminated, it can be divided into two categories: single trajectory compression and multiple trajectory compression.

A. Single Trajectory Compression

Trajectory simplification technology is the primary form of eliminating intra-trajectory redundancy. According to the error metrics, it can be divided into position-preserving, direction-preserving, and speed-preserving trajectory simplification algorithms. Since the distance is the most intuitive measure between points, most studies aim to simplify trajectories by distance-based error metrics, thereby preserving the location information of trajectories [8], [9], [19], [20], [21]. Douglas–Peucker (DP) [8] is a heuristic algorithm that ensures the maximum Perpendicular euclidean Distance (PED) error of the simplified trajectory is limited within a given threshold. Top-Down Time-Ratio (TD-TR) [9] fully considers the spatio-temporal characteristics on the basis of DP, and adopts Synchronous euclidean Distance (SED) as a metric. DP and TD-TR are widely praised for their high-quality simplification [22]. Recently, various extensions of the DP algorithm have been proposed. These include integrating speed features to preserve essential motion characteristics [23], introducing adaptive compression thresholds with feature point selection to enhance trajectory fidelity [24], and determining thresholds based on area difference between the original and simplified trajectories [25]. To achieve real-time compression, sliding window techniques are also used in trajectory simplification [11], [26], [27], [28], [29], [30], [31]. Since speed and direction are also crucial characteristics of moving objects, corresponding speed-preserving [32] or direction-preserving trajectory simplification algorithms are developed [6], [10], [12], [33]. These advancements in single-trajectory compression techniques improve the efficiency of trajectory-based applications by reducing data size while maintaining key motion characteristics, with specific applications in areas such as road network analysis [21], [32] and vessel navigation pattern recognition [24], [25].

Although the trajectory simplification techniques mentioned above exhibit high compression performance, most of them rely on their own chosen error metrics, which leads to a large loss of information on other features [13]. In fact, when using decompressed trajectory data for analysis, the information loss should be comprehensive rather than performing well in one aspect and performing poorly in other aspects.

B. Multiple Trajectory Compression

Compared to the STC algorithm, the MTC algorithm further reduces storage overhead by eliminating inter-trajectory redundancy [14] and enhances location-based services by exploiting inter-trajectory correlations, which further improves data

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

processing efficiency and enables more accurate analysis [15], [16], [17], [18], [34], [35]. TrajStore [15] performs trajectory clustering by calculating the similarity between trajectories. Trajectories with high similarity are grouped, and a reference trajectory is designated for each group. All trajectory points of the reference trajectory are stored in full. For the spatial data of the non-reference trajectory within the group, its time mapping value on the reference trajectory is used instead. Trajic [34] improves on TrajStore's encoding method, but still uses the same principle of multiple trajectory compression. SMTC [16] combines the STC algorithm and time mapping to reduce redundancy by storing time pairs in similar trajectories. However, the time mapping technology has a large time overhead for MTC, and also introduces the problem of reference dependence.

REST [17], [18] is a data-driven framework that compresses trajectories by constructing a large set of reference sub-trajectories. However, it overlooks the redundancy of the trajectory itself. In actual compression, matching reference trajectories entail a considerable time overhead, and achieving a higher compression ratio demands a large error threshold.

In addition to reference-based clustering, another method exists that leverages incremental computation and motion trend modeling to assess the importance of trajectory points between trajectories, effectively eliminating redundant data while preserving essential information. However, retaining key data can be difficult when dealing with extreme anomalies (e.g., sharp turns) or complex motion patterns (e.g., oscillations or slow motion) [35].

III. Intra-Trajectory Error Balancing

The main challenge of simplifying a single trajectory is how to determine whether a trajectory point belongs to a feature point or a redundant point. Existing trajectory simplification algorithms rely on a single error metric, which often results in polarized feature losses [13]. Therefore, we propose a method to remove redundancy within the trajectory. This improves the performance of the algorithm on the comprehensive error and the practical application ability. Sections III-A, III-B, and III-C introduce the definition of the score, the resolution of the *Min-Error* problem, and the implementation details of the algorithm, respectively. Table I gives the important notations and their definitions as used in this study.

A. Score Definition

Each trajectory T consists of several trajectory points $T = \{P_1, P_2, \ldots, P_n\}$, and each point P_i is usually a triple entry $P_i(x_i, y_i, t_i)$, including spatial information, such as longitude and latitude, and time information. The information loss of trajectory simplification is mainly reflected in the three characteristics of position, speed, and direction. Therefore, commonly used error metrics are PED, SED, speed, and direction [36]. As shown in Fig. 1, we take the DP [8] algorithm as an example to analyze in detail its polarization of feature loss. A large feature loss means that the recovered trajectory data is of low quality or even unusable, which we should avoid.

TABLE I
Important Notations in This Study

Notation	Definition
$T = \{P_1, P_2, ..., P_n\}$	A trajectory with size n
$T' = \{P_{s_1}, P_{s_2}, ..., P_{s_m}\}$	The simplification of trajectory T ($m \leq n$)
$\|T\|$	Size of trajectory T
$P_i = (x_i, y_i, t_i)$	The ith trajectory point of trajectory T
$\overrightarrow{P_iP_j}$	Trajectory segment from P_i to P_j ($i < j$)
$P'_i = (x'_i, y'_i, t'_i)$	Estimated position of P_i at time
$\theta_{i,j}$	Angle of $\overrightarrow{P_iP_j}$ (in radians)
$\Delta(\theta_{i,i+1}, \theta_{s,e})$	Direction difference between $\overrightarrow{P_iP_{i+1}}$ and $\overrightarrow{P_sP_e}$
$W = [w_1, w_2, w_3, w_4]$	Weight of PED, SED, SAD and DAD
ω	Store budget
$\epsilon(T')$	Error of simplified trajectory T'
$TG = \{T'_1, T'_2, ..., T'_n\}$	Multiple similar trajectories after simplification
$P_j^i = (x_j^i, y_j^i, t_j^i)$	The jth trajectory point of the ith trajectory
Clu_i	The ith Clusters containing valid feature points
$C_i = (x_i^c, y_i^c)$	Fused central position point of a cluster
μ	Maximum distance of feature points in a cluster
α	Accumulation coefficient
μ_r	Revised Distance Threshold

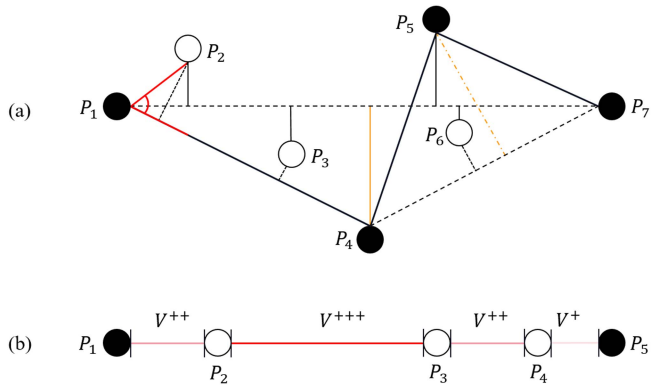


Fig. 1. Douglas–Peucker algorithm. (a) Direction error of DP algorithm. (b) Speed error of DP algorithm.

In Fig. 1(a), the DP algorithm first connects the starting point P_1 and the end point P_7, and then calculates the PED distance from the points in the middle of P_1 to P_7 to the straight line P_1P_7. The trajectory point with the largest PED distance that exceeds the distance error threshold is selected as the feature point for segmentation. Next, DP repeats the above steps for trajectory segments P_1P_4 and P_4P_7. Finally, the feature points P_1, P_4, P_5, and P_7 are retained. Although the PED distance from P_2 to P_1P_4 is smaller than the threshold, the angle value of $\angle P_2P_1P_4$ is actually close to 90°. This leads to a large direction error. Therefore, the DP algorithms using PED as a metric perform poorly on direction error. In Fig. 1(b), since P_1, P_2, P_3, P_4, and P_5 are on a straight line, the DP algorithm omits all the middle trajectory points P_2, P_3, and P_4. However, when the acquisition frequency does not change much, the speed between P_2P_3 is significantly higher than that of P_1P_2, P_3P_4 and P_4P_5. Therefore, the DP algorithms using PED as a metric perform poorly on speed error.

In order to make the simplified trajectory perform better on the comprehensive error rather than a single metric, we represent the importance of trajectory points in the form of a score, that is

$$Score(P_i, \overrightarrow{P_sP_e}) = \varphi(\text{PED}_i, \text{SED}_i, \text{SAD}_i, \text{DAD}_i), \quad (1)$$

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

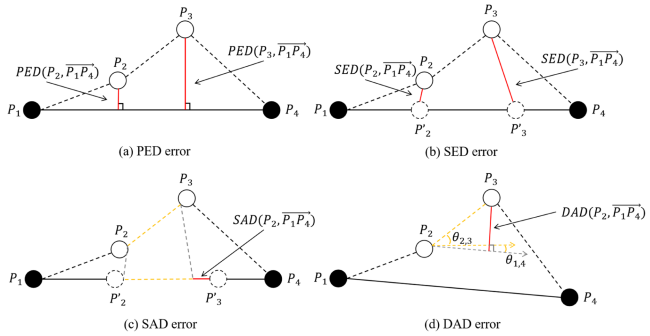


Fig. 2. An example of PED, SED, DAD, and SAD errors.

where $\overrightarrow{P_sP_e}$ represents the simplified trajectory segment where the trajectory point P_i is located. P_s and P_e represent the start point and end point of the segment $\overrightarrow{P_sP_e}$. As shown in Fig. 2(a), the PED error between the trajectory point P_i and the simplified trajectory segment $\overrightarrow{P_sP_e}$ is denoted by $\text{PED}(P_i, \overrightarrow{P_sP_e})$, which is defined as the vertical distance error caused by removing P_i. Then, we have

$$PED(P_i, \overrightarrow{P_sP_e}) = \frac{|(y_e - y_s) \times x_i + y_i \times (x_s - x_e) + y_s \times x_e - x_s \times y_e|}{\sqrt{(y_e - y_s)^2 + (x_e - x_s)^2}}. \quad (2)$$

As shown in Fig. 2(b), the SED error between point P_i and segment $\overrightarrow{P_sP_e}$ is denoted by $\text{SED}(P_i, \overrightarrow{P_sP_e})$, which is defined as the distance difference between P_i and its estimated position P'_i at time t_i on segment $\overrightarrow{P_sP_e}$. The SED error can be calculated by

$$\text{SED}(P_i, \overrightarrow{P_sP_e}) = \sqrt{(x_i - x'_i)^2 + (y_i - y'_i)^2}, \quad (3)$$

where x'_i and y'_i of the estimated position $P'_i(x'_i, y'_i, t_i)$ can be calculated by

$$\begin{cases} x'_i = x_i + \frac{x_e - x_s}{t_e - t_s} \times (t_i - t_s) \\ y'_i = y_i + \frac{y_e - y_s}{t_e - t_s} \times (t_i - t_s) \end{cases}. \quad (4)$$

Since the units of measurement for speed error and direction error are different from those of PED and SED, this is detrimental to the calculation of the comprehensive score. Therefore, we convert the speed error and direction error into Speed-Aware Distance (SAD) error and Direction-Aware Distance (DAD) error, as shown in Fig. 2(c) and (d). The SAD error between the trajectory point P_i and the simplified trajectory segment $\overrightarrow{P_sP_e}$ is denoted by $\text{SAD}(P_i, \overrightarrow{P_sP_e})$, which is defined as the product of the speed difference between $\overrightarrow{P_iP_{i+1}}$ and $\overrightarrow{P_sP_e}$ and the time difference between P_i and P_{i+1}; it is calculated by

$$\begin{aligned} \text{SAD}(P_i, \overrightarrow{P_sP_e}) &= |V(P_i, P_{i+1}) - V(P_s, P_e)| \times (t_{i+1} - t_i) \\ &= |Dist(P_i, P_{i+1}) - Dist(P'_i, P'_{i+1})|, \end{aligned} \quad (5)$$

where $V(P_i, P_{i+1})$ represents the average speed of the trajectory point P_i, that is, $V(P_i, P_{i+1}) = Dist(P_i, P_{i+1})/(t_{i+1} - t_i)$. $Dist(P_i, P_j)$ represents the linear distance between trajectory point P_i and P_j, that is, $Dist(P_i, P_j) = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$. The DAD error between the trajectory point P_i and the simplified trajectory segment $\overrightarrow{P_sP_e}$ is denoted by $\text{DAD}(P_i, \overrightarrow{P_sP_e})$, which is defined as the product of the sine value of the direction difference between $\overrightarrow{P_iP_{i+1}}$ and $\overrightarrow{P_sP_e}$ and the distance of $\overrightarrow{P_sP_e}$; therefore, we have

$$\text{DAD}(P_i, \overrightarrow{P_sP_e}) = Dist(P_i, P_{i+1}) \times \sin(\Delta(\theta_{i,i+1}, \theta_{s,e})), \quad (6)$$

where direction $\theta_{i,j}$ is defined as the angle (in radians) between the trajectory segment $\overrightarrow{P_iP_j}$ and the horizontal line, that is, $\theta_{i,j} = arctan((y_j - y_i)/(x_j - x_i))$. The direction difference of trajectory segments $\overrightarrow{P_iP_{i+1}}$ and $\overrightarrow{P_sP_e}$, denoted by $\Delta(\theta_{i,i+1}, \theta_{s,e})$ [12], is defined as the smaller one between the angle rotated counterclockwise from $\overrightarrow{P_iP_{i+1}}$ to $\overrightarrow{P_sP_e}$ and the angle rotated counterclockwise from $\overrightarrow{P_sP_e}$ to $\overrightarrow{P_iP_{i+1}}$, that is, $\Delta(\theta_{i,i+1}, \theta_{s,e}) = min\{|\theta_{i,j} - \theta_{s,e}|, 2\pi - |\theta_{i,j} - \theta_{s,e}|\}$.

In order to adapt to the trajectory data storage requirements in different scenarios, we specify the dot product of the weight $W = [w_1, w_2, w_3, w_4]$ and the errors of PED, SED, SAD, and DAD as the comprehensive score. At the same time, in order to ensure the validity of the weight, we use the arctangent transformation in nonlinear normalization to narrow the difference between the error metrics, that is

$$F(x) = \frac{2}{\pi} \times \arctan x. \quad (7)$$

Therefore, the score of the trajectory point P_i in (1) is finally expressed as

$$Score(P_i, \overrightarrow{P_sP_e}) = W \cdot F([\text{PED}_i, \text{SED}_i, \text{SAD}_i, \text{DAD}_i]).$$

The higher the comprehensive score of a trajectory point, the greater the feature loss caused by removing it, and therefore it should be retained. Then, we get the score trajectory point P_{target} in the simplified trajectory segment $\overrightarrow{P_sP_e}$ by (8).

$$P_{\text{target}} = MaxScore(s, e) = \max_{s<i<e} Score(P_i, \overrightarrow{P_sP_e}). \quad (8)$$

B. Min-Error Problem

The *Min-Size* problem aims to minimize the number of retained feature points under the given error threshold, which is only applicable to the situation where users have a precise understanding of fault tolerance. In fact, the opposite is often the case. The *Min-Error* problem, as the dual problem of the *Min-Size* problem, handles this situation well [12].

We simplify the given trajectory $T = \{P_1, P_2, \ldots, P_n\}$ to get the simplified trajectory $T' = \{P_{s_1}, P_{s_2}, .., P_{s_m}\}$, where $m \leq n$, $1 = s_1 < s_2 < \ldots < s_m = n$ and $m = \lfloor \omega \times n \rfloor$. The storage budget, denoted by ω, is defined as the number m of retained feature points specified by the user divided by the number n of original trajectory points. We define the size of a trajectory T as the number of trajectory points included in T, denoted by $|T|$. We denote the comprehensive error of the simplified trajectory T' as $\epsilon(T')$. Then, the *Min-Error* problem aims to

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

simplify the trajectory T according to the given storage budget ω, so that the comprehensive error $\epsilon(T')$ of the simplified trajectory T' is minimized. The formal definition is as follows.

Min-Error Problem: Given a trajectory T and a storage budget ω, we find a simplification T' such that $|T'| \leq \omega \times |T|$ and $\epsilon(T')$ are minimized.

We evaluate the amount of information carried by a trajectory point through score (Section III-A). Then, we retain the point with the high score as a feature point each time and update the score of other trajectory points on the two segments divided by this point, until the storage budget is reached. Under the same storage budget, the *Min-Error* problem guarantees that the error of simplified trajectory T'_1 is consistent with the error of the simplified trajectory T'_2 corresponding to the *Min-Size* problem, namely $\epsilon(T'_1) = \epsilon(T'_2)$. Next, we propose the theorem and prove its correctness.

Theorem 1: For the same trajectory T, under the same storage budget (or compression ratio), $\epsilon(T'_1)$ corresponding to the Min-Error problem is equal to $\epsilon(T'_2)$ corresponding to the Min-Size problem.

Proof: Consider the weight vector $W = [1, 0, 0, 0]$, where the SEB algorithm scores and selects trajectory points based only on the PED error. At this point, the behavior of the SEB algorithm is the same as that of the DP algorithm, both of which select the point with the largest PED error as the next feature point to be retained.

Assume that under the same trajectory T and the same storage budget ω, the simplified trajectory generated by the SEB algorithm is T'_1, the simplified trajectory generated by the DP algorithm is T'_2, and $T'_1 \neq T'_2$.

That is, there is at least one point P_{s_i} that belongs to T'_1 but not to T'_2.

Since the DP algorithm selects points to be retained based on the PED error, and the SEB algorithm behaves consistently under this weight setting, for all points retained by the SEB algorithm, their PED errors exceed the threshold τ set by the DP algorithm. In particular, point P_{s_i} satisfies $\text{PED}(P_{s_i}, \overrightarrow{P_s P_e}) > \tau$, so according to the rules of the DP algorithm, P_{s_i} should be retained in T'_2.

However, we assume that P_{s_i} does not belong to T'_2, which contradicts the above conclusion that P_{s_i} should belong to T'_2. Therefore, the initial assumption $T'_1 \neq T'_2$ does not hold.

In summary, under the weight vector $W = [1, 0, 0, 0]$, the simplified trajectories T'_1 and T'_2 generated by the SEB algorithm and the DP algorithm satisfy $T'_1 = T'_2$ and $\epsilon(T'_1) = \epsilon(T'_2)$.

C. The SEB Algorithm

In this section, we propose the SEB algorithm (Algorithm 1) to balance the error caused by trajectory simplification and solve the *Min-Error* problem. We use the idea of divide and conquer to find the trajectory points with the local highest score, and use the priority queue to obtain the trajectory points with the global highest score as feature points. The specific description of the SEB algorithm is as follows.

Algorithm 1: The SEB Algorithm.

Input: $T = \{P_1, P_2, ..., P_{\text{n}}\}, \omega$
Output: $T' = \{P_{\text{s}_1}, P_{\text{s}_2}, ..., P_{\text{s}_{\text{m}}}\} (m \leq n, 1 = s_1 < s_2 < ... < s_{\text{m}} = n)$

1 Initialize the priority queue PQ;
2 Initialize $T' \leftarrow \varnothing$ and append P_1 and P_{n} to T';
3 $start \leftarrow 1; end \leftarrow n; P_{\text{target}} \leftarrow null$;
4 $m \leftarrow \lfloor \omega \times n \rfloor$;
5 **for** $(i = 1; i \leq m - 2; i++)$ **do**
6 **if** P_{target} *is null* **then**
7 // obtain the highest score trajectory point on the entire trajectory by Eq. (8)
8 $P_{\text{target}} \leftarrow MaxScore(start, end)$;
9 **else**
10 // take out a trajectory point from PQ
11 $P_{\text{target}} \leftarrow PQ.out()$;
12 **end**
13 **end**
14 Append P_{target} to T';
15 // Obtain $start$ and end index of the simplified segment where P_{target} is located
16 $(start, end) \leftarrow IndexOf(P_{\text{target}})$
17 // update the score
18 $P_{\text{left}} \leftarrow MaxScore(start, target)$;
19 $P_{\text{right}} \leftarrow MaxScore(target, end)$;
20 Append P_{left} and P_{right} to PQ;
21 return T';

1) In Lines 1-4, the priority queue PQ, the reduced trajectory T' and the size m of T' are initialized. P_1 and P_{n} must be retained in T' because they represent the start and end of the trajectory.
2) In Lines 5-12, for the remaining $m - 2$ feature points, SEB initially takes the entire trajectory as a segment and looks for the highest local score trajectory point in the segment. SEB calculates the PED, SED, SAD, and DAD errors of each trajectory point in the segment by (2)-(6), and then performs transformation by (7). Finally, W is weighted to find the trajectory point P_{target} with the highest score in the segment. Otherwise, it takes the first element from the priority queue PQ to P_{target}.
3) In Lines 14-20, P_{target} is put into T' as the feature point with the highest global score. Next, the trajectory segment is divided into two new sub-segments by P_{target}. Then, the scores of trajectory points in two sub-segments are updated, and the obtained two trajectory points with the highest local score are put into the PQ.

The SEB algorithm performs trajectory simplification according to a given storage budget. For a trajectory with N trajectory points and a storage budget of ω, a total of $\omega \times N$ iterations are required. The time complexity of updating the score for each iteration is $O(N)$. The priority queue stores up to $\omega \times N - 1$ elements and the time complexity of adding or removing an element each time is $log(\omega \times N)$. Therefore, the time complexity of the SEB algorithm is $O(\omega \times N^2 \times log(\omega \times N)) = O(M \times N \times logM)$, where $M = \omega \times N$ represents the number of feature points to be stored.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

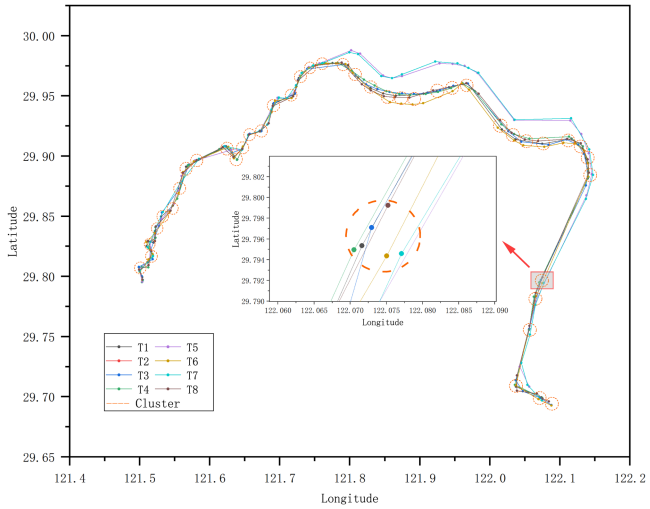


Fig. 3. Schematic diagram of cluster distribution of feature points in similar trajectories.

IV. Inter-Trajectory Feature Point Clustering and Fusion

This section describes how we extract commonality between similar trajectories by reducing the granularity of clustering from trajectory segments to feature points. This approach breaks through the performance bottleneck of the existing MTC algorithms, reduces the similarity requirements of multiple trajectories, and avoids the problem of trajectory dependence. Sections IV-A, IV-B, and IV-C introduce the core idea, the error accumulation problem, and the implementation details of the algorithm, respectively.

A. Core Idea

The starting point of our algorithm is the cluster characteristics exhibited by the feature points of similar trajectories. For the 8 similar trajectories (after SEB simplification) given in Fig. 3, the feature points in the cluster are very close in spatial position. Therefore, clustering the feature points of multiple similar trajectories can eliminate redundancy between trajectories to the greatest extent.

Given a similar trajectory group $TG = \{T_1, T_2, \ldots, T_m\}$, we cluster all feature points contained in TG according to the distance threshold μ. In detail, initially, we treat each feature point as a separate cluster. We define the inter-cluster distance as the maximum distance between any two feature points in two clusters, denoted by $CDist(Clu_i, Clu_j)$, which is calculated by

$$CDist(Clu_i, Clu_j) = \max_{P_k \in Clu_i, P_l \in Clu_j} Dist(P_k, P_l), \quad (9)$$

where P_k and P_l are the feature points in Clu_i and Clu_j respectively. When the inter-cluster distance is less than the given threshold, that is, $CDist(Clu_i, Clu_j) < \mu$, the two clusters are merged into a new cluster. Finally, each cluster is fused with a central point containing spatial information, denoted by $C_i = (x_i^c, y_i^c)$, which is defined as the common spatial information of each feature point in a cluster. Therefore, we have

$$\begin{cases} x_i^c = \frac{1}{k} \times \sum_{j=1}^{k} x_j (1 < k \le m) \\ y_i^c = \frac{1}{k} \times \sum_{j=1}^{k} y_j (1 < k \le m) \end{cases}, \quad (10)$$

where k represents the number of effective feature points in a cluster, and its maximum is the number m of trajectories contained in the trajectory group TG. We consider a feature point to be valid if there is only one unique feature point per trajectory in a cluster. The error caused by the fusion of effective feature points is bounded, which is proved in Theorem 2. Then, we replace the spatial information of other effective feature points in a cluster with the central point C_i. Finally, Huffman coding is performed on redundant information to reduce storage overhead. An example diagram is shown in Fig. 4.

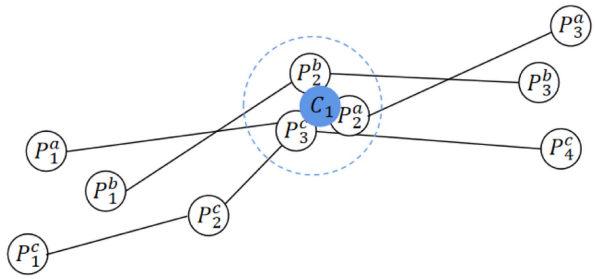


Fig. 4. Illustrative example of the core idea.

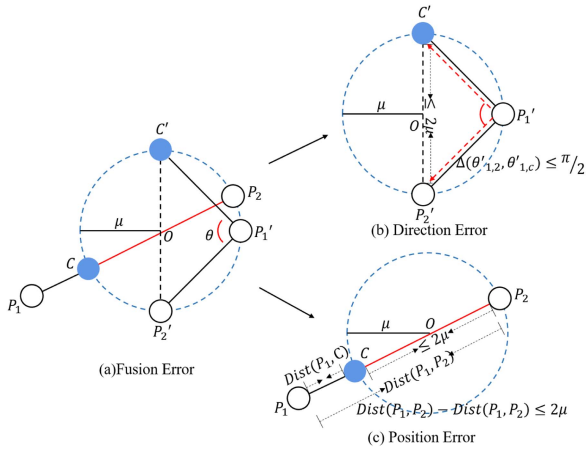


Fig. 5. An example about the boundedness of fusion error.

Theorem 2: The error caused by the fusion of effective feature points in the same cluster is bounded.

Proof: Taking Fig. 5 as an example, we assume the clustering distance threshold is $2 \times \mu$.

We take the trajectory segment $\overrightarrow{P_1P_2}$ as an example to prove the boundedness of the position error (Fig. 5(c)). When P_2 forms a cluster with other valid feature points, the center position point is located at C in the most extreme case. At this time, $Dist(P_2, C)$ is infinitely close to the diameter of the circle O, that is, $Dist(P_2, C) \approx 2 \times \mu$. If the central position C is used to replace the spatial information of P_2, then the introduced

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

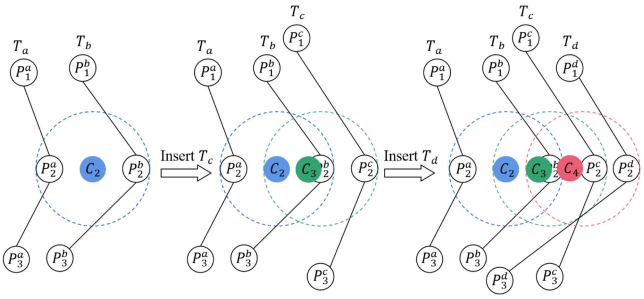


Fig. 6. The error accumulation problem of CFPF algorithm.

position error is $Dist(P_1, P_2) - Dist(P_1, C)$, which is less than $2 \times \mu$. Since the time information of trajectory point P_2 remains unchanged, when the position error is bounded, it means that the speed error is also bounded. Then, we take the trajectory segment $\overrightarrow{P_1'P_2'}$ as an example to prove the boundedness of the direction error (Fig. 5(b)). When other effective feature points of P_2' are gathered into a cluster, the central position point is located at C' in the most extreme case. At this time, $\Delta(\theta'_{1,2}, \theta'_{1,c})$ is infinitely close to $\pi/2$ (the angle subtended by the diameter is a right angle). If the central position C' is used to replace the spatial information of P_2', then the introduced direction error is $\Delta(\theta'_{1,2}, \theta'_{1,c})$, which is smaller than $\pi/2$. Therefore, the error caused by the fusion of valid feature points in the same cluster is bounded. In addition, the simplified trajectory discards a large number of redundant points, and the distance threshold μ of clustering is set to be small. Therefore, the error is much lower than that caused by the extreme cases described above.

B. The Error Accumulation Problem

We found that, in extreme cases, the clustering-based feature point fusion algorithm has a potential error accumulation problem, which is usually called Concept Drift in clustering algorithms [37], as shown in Fig. 6. In simple terms, as trajectories are continuously inserted, we continue to cluster based on the previous clustering results. However, the actual distance error may continue to increase beyond the error threshold we set. Note that the extreme case here means that the feature points of each new cluster are close to the edge of the cluster range and the feature points are all valid. $T(n)$ represents the distance from the nth cluster center to the first feature point. Then, as the number of feature points increases to n, we have

$$T(n) = \frac{1}{n} \times (T(n-1) + \mu + (n-1) \times T(n-1)), \quad (11)$$

where μ represents the given distance threshold. The above formula can be simplified to $T(n) = T(n-1) + \frac{\mu}{n}$. When there is only one feature point in the cluster, such as P_2^{a} in Fig. 6, it is the center point itself, that is, $T(1) = 0$. When adding the second extreme feature point P_2^{b}, the position of the center point C_2 is between the two feature points, that is, $T(2) = \frac{1}{2} \times \mu$. When the third extreme feature point P_2^{c} is added, the distance between the cluster center C_3 and P_2^{a} has reached $\frac{5}{6} \times \mu$, that is, $T(3) = \frac{5}{6} \times \mu$. When the fourth extreme feature point P_2^{d} is added, the distance $T(4)$ between the cluster center C_4 and P_2^{a} has reached $\frac{13}{12} \times \mu$, exceeding the preset distance threshold μ.

With the addition of extreme feature points, the central position point generated by fusion may deviate more and more from the first feature point. This means that clustering new simplified trajectories may lead to error accumulation problems, which may exceed our pre-set distance threshold in extreme cases. Therefore, we provide a scheme to solve the problem of error accumulation. We set an accumulation coefficient and use a modified distance threshold to limit the accumulation of distance errors. The accumulation coefficient is a parameter related to the number of expected trajectories, denoted by α. We have

$$\alpha = \frac{1}{ln(n) - 1} (0 < \alpha \leq 1), \quad (12)$$

where n represents the number of trajectories expected to be stored. The revised distance threshold is a calculated value smaller than the distance threshold μ, denoted by μ_{r}, and defined as the product of the accumulation coefficient and the distance threshold. Therefore, we have

$$\mu_{\mathrm{r}} = \alpha \times \mu. \quad (13)$$

Using the revised distance threshold solves the problem of error accumulation, even in extreme cases, the final cumulative distance error does not exceed the given distance threshold. We proved this in Theorem 3.

Theorem 3: Using the revised distance threshold μ_{r} ensures that the cumulative distance error does not exceed the given distance threshold μ in extreme cases.

Proof: According to (11), (12), and (13), we have $T_{\mathrm{n}} = T_{n-1} + \frac{\mu_{\mathrm{r}}}{n}$, where $\mu_{\mathrm{r}} = \alpha \times \mu$. Then, $T_{\mathrm{n}} = T_{n-2} + \alpha \times \frac{\mu}{n-1} + \alpha \times \frac{\mu}{n}$. By expanding the above formula, we get $T_{\mathrm{n}} = T_1 + \alpha \times \frac{\mu}{2} + \alpha \times \frac{\mu}{3} + \cdots + \alpha \times \frac{\mu}{n}$. Then, $T_{\mathrm{n}} + \alpha \times \mu = \alpha \times \mu \times (1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}) = \alpha \times \mu \int_1^{\mathrm{n}} \frac{1}{x} dx$. We solve the integral and further get $T_{\mathrm{n}} = \alpha \times \mu \times ln(n) - \alpha \times \mu$. If we want T_{n} to be less than or equal to μ, we only need to make $\alpha \leq \frac{1}{ln(n)-1}$. Therefore, when the accumulation coefficient $\alpha = \frac{1}{ln(n)-1}$, the revised distance threshold μ_{r} ensures that the maximum distance error of a cluster is within μ.

C. The CFPF Algorithm

In this section, we propose the CFPF algorithm (Algorithm 2) to extract commonalities among multiple similar trajectories, thereby further reducing the storage overhead. The specific description of the CFPF algorithm is as follows.

1) In Lines 4–7, each trajectory in the TG is simplified using the SEB algorithm. Then the feature points in each simplified trajectory are treated as a separate cluster and added to $ClusterSet$.
2) In Lines 11–15, CFPF calculates the distance between two clusters by (9), and compares it with the revised distance threshold to determine whether clusters need to be merged. The more effective feature points in a cluster, the higher the compression benefit.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: The CFPF Algorithm.

Input: $TG = \{T_1, T_2, ..., T_m\}$, μ
Output: The final compressed data file $DataFile$

1 Initialize the cluster set as $ClusterSet \leftarrow \varnothing$;
2 Calculate the coefficient α using Eq. (12);
3 Calculate the revised error threshold μ_r using Eq. (13);
4 **for** *(each T in TG)* **do**
5 $T' = SEB(T)$;
6 **for** *(each P in T')* **do**
7 Add P to $ClusterSet$;
8 **end**
9 **end**
10 **for** *(each Clu_i in $ClusterSet$)* **do**
11 **for** *(each Clu_j in $ClusterSet$)* **do**
12 // calculate the cluster distance using Eq. (9)
13 $d \leftarrow CDist(Clu_i, Clu_j)$;
14 **if** *($d \leq \mu_r$)* **then**
15 $Clu_i \leftarrow Clu_i \cup Clu_j$;
16 **end**
17 **end**
18 Calculate C_i of Clu_i using Eq. (10);
19 **for** *(each P in Clu_i)* **do**
20 Update P using C_i;
21 **end**
22 **end**
23 $DataFile \leftarrow$ Huffman(TG);
24 return $DataFile$;

3) In Lines 18–20, CFPF obtains the central point of the cluster by (10), and uses it as the spatial information of all valid feature points in the cluster.
4) In Line 23, Huffman encoding is performed at the end to obtain the maximum compression benefit.

The CFPF algorithm demonstrates high computational efficiency in trajectory processing, which we confirm through a detailed analysis of its time and space complexity. The algorithm consists of three major steps: trajectory simplification, feature point clustering and fusion, and encoding.

In the time complexity analysis, the trajectory simplification step simplifies K trajectories using the SEB algorithm with a complexity of $O(KMN \log M)$, where N is the average length of each original trajectory, and M is the average length of each simplified trajectory ($M \ll N$). The feature point clustering, fusion, and encoding steps process the simplified trajectories with a complexity of $O(K^2 \tilde{} M^2 + \log(KM))$. Consequently, the overall time complexity of the CFPF algorithm is $O(KMN \log M + K^2 \tilde{} M^2 + \log(KM))$.

In the space complexity analysis, the primary storage requirement involves maintaining feature point group information, with a complexity of $O(KM)$. To further optimize storage, Huffman encoding compresses redundant data using an (l, v) format, where l represents the length of the information and v is the encoded value. These techniques significantly reduce storage overhead while maintaining computational efficiency.

TABLE II
DATASETS' INFORMATION

Data sets	Frequency	Number of points	Size
Berlin_0.05	1s∼3s	10*1000	373KB
Walking	5s∼15s	10*200	56.5KB
Airline	20s∼60s	20*200	112KB
Illinois	1s	220*2200	13.3MB
T-Drive	177s	15 million	752MB

V. EXPERIMENT

In this section, we evaluate our proposed method and the related trajectory compression algorithm in Section II using five different real datasets. All experiments were performed on Windows 10 operating system with 8 G memory, 1 TB disk space, and Intel(R) Core(TM) i5-10500 CPU @ 3.10 GHz central processing unit. In the evaluation experiment of the multiple trajectory compression algorithms, for the sake of fairness, we perform Huffman coding on the results of other algorithms and compare the actual space occupied by the final compressed file.

A. Datasets Description

The characteristics of trajectory data may vary greatly depending on how the object moves. Therefore, we use five different trajectory datasets to check whether the algorithm is robust to different motion patterns. The statistics and visualizations of five datasets are shown in Table II and Fig. 7, respectively.

- *Berlin_0.05:*[1] The public dataset contains trajectories of simulated cars on the streets of Berlin, the capital of Germany. From these, we select 10 similar trajectories, and each trajectory contains 1000 trajectory points.
- *Walking:* The dataset contains our real walking trajectory from home to the laboratory. Each trajectory contains about 200 trajectory points, and a total of 10 trajectory data are collected.
- *Airline:*[2] The dataset contains 20 flight trajectories of aircraft CCA1740 collected from. Each trajectory contains about 200 trajectory points.
- *Illinois:*[3] The dataset is obtained from two members of Argonne National Laboratory of UIC during their daily commute for six months. Each trajectory represents a continuous trip of a member in Cook county and/or Dupage county of Illinois. The trajectories are sampled strictly every second.
- *T-drive:*[4] This dataset contains the GPS trajectories of 10357 taxis during the period of Feb. 2 to Feb. 8, 2008 within Beijing. The total number of points in this dataset is about 15 million and the total distance of the trajectories reaches to 9 million kilometers. The average sampling

[1] Berlin_0.05 Dataset, 2007, Available at: https://secondo-database.github.io/BerlinMOD/BerlinMOD.html
[2] Airline Dataset, 2022, Available at: https://zh.flightaware.com/live
[3] Illinois Dataset, 2006, Available at: https://www.cs.uic.edu/~boxu/mp2p/gps_data.html
[4] T-Drive trajectory data sample, 2011, Available at: https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample/

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

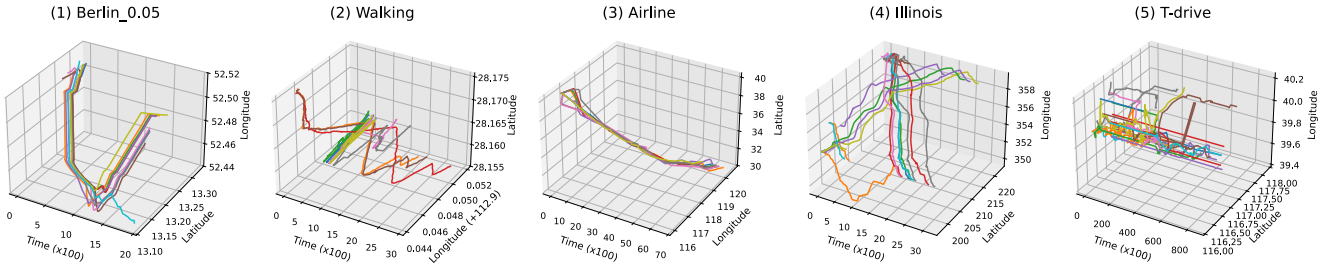


Fig. 7. Visualization of motion trajectories in five datasets.

interval is about 177 seconds with a distance of about 623 meters.

B. Metrics

We evaluate the performance of our proposed algorithm using compression ratio (CR), average PED error, average SED error, average speed error, average direction error, average dynamic time warp (DTW) distance, compression time overhead, and recovery time overhead. They are defined as follows.

Definition 1: Compression ratio. The degree of data reduction by the compression algorithm is defined as the ratio of the original data size to the compressed data size.

$$\mathrm{CR} = \frac{Size_{\mathrm{o}}}{Size_{\mathrm{c}}},$$

where $Size_{\mathrm{c}}$ represents the compressed data file size and $Size_{\mathrm{o}}$ represents the uncompressed original data set size.

Definition 2: Average PED error. The PED distance is used to evaluate the error of the compressed trajectory in the vertical distance. The perpendicular distance between the trajectory point P_{i} and the trajectory segment $\overrightarrow{P_{\mathrm{s}}P_{\mathrm{e}}}$ is their shortest distance, which is calculated by (2).

Definition 3: Average SED error. The SED distance considers the time dimension on the basis of the PED distance. It is defined as the distance between the trajectory point P_{i} and its time synchronization estimated point P'_{i}, which is calculated by (3).

Definition 4: Average speed error. The speed error of trajectory point P_{i} is defined as the difference between the speed of P_{i} and the speed of the simplified trajectory segment $\overrightarrow{P_{\mathrm{s}}P_{\mathrm{e}}}$ where it is located. We have

$$Error_{\mathrm{speed}}(P_{\mathrm{i}}, \overrightarrow{P_{\mathrm{s}}P_{\mathrm{e}}}) = |V(P_{\mathrm{i}}, P_{\mathrm{i+1}}) - V(P_{\mathrm{s}}, P_{\mathrm{e}})|.$$

Definition 5: Average direction error. The direction error of a trajectory point P_{i} is defined as the direction difference between $\overrightarrow{P_{\mathrm{i}}P_{\mathrm{i+1}}}$ and the simplified trajectory segment $\overrightarrow{P_{\mathrm{s}}P_{\mathrm{e}}}$ where it is located, denoted as $\Delta(\theta_{\mathrm{i,i+1}}, \theta_{\mathrm{s,e}})$, see Section III-A.

Definition 6: Average DTW distance. The DTW distance is often used to measure the similarity between two time series of the same or different lengths. Trajectories T_{a} and T_{b} contain m and n trajectory points, respectively, and these sequences form a matrix $M = T_{\mathrm{a}} \times T_{\mathrm{b}}$, where point $M_{\mathrm{i,j}}$ is the distance between $P^{\mathrm{a}}_{\mathrm{i}}$ and $P^{\mathrm{b}}_{\mathrm{j}}$. DTW finds the shortest path W between $M_{0,0}$ and $M_{m,n}$, and the sum of the distances of the edges that W passes through is used as the evaluation criterion for the similarity of trajectories T_{a} and T_{b}. The smaller the total distance, the higher the similarity. Therefore, we have

$$Error_{\mathrm{dtw}} = \sum_{\mathrm{i=0}}^{\mathrm{k}} W_{\mathrm{i}},$$

where k is the number of elements in W. More details on optimal path finding can be found in [38]. Fig. 8

illustrates an example of the DTW algorithm, we can see that the edge connecting T_{a} and T_{b} forms the optimal path W.

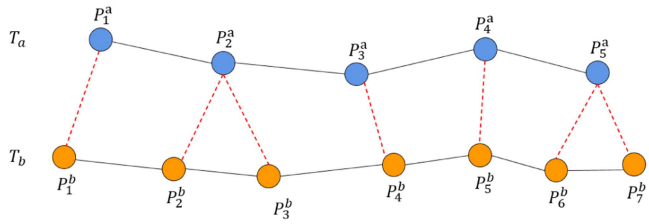


Fig. 8. Example of DTW distance between trajectories A and B.

C. Performance of SEB Algorithm

In this section, we use classical DP [8] and TD-TR [9] algorithms as baseline algorithms to evaluate the performance of our SEB algorithm on comprehensive error loss. In order to ensure the validity of the results, we have carried out necessary preprocessing on the original trajectory data, including eliminating trajectory points with missing partial information and completely repeated and extremely abnormal data. We simplify several trajectories for each dataset individually and take the mean of the error as the result. Without special instructions, the parameter W of the SEB algorithm is set to [0.3,0.5,0.1,0.1] by default (as shown in Fig. 9).

Fig. 9 shows the changes in the comprehensive scores after testing 7 different weight W strategies on 5 different data sets. As can be seen from the results in the figure, the scores of schemes a, b, c, and d are significantly higher than those of schemes e, f, and g, which means that in this experiment, the optimal weight distribution tends to emphasize the impact of PED error and SED error. After further comparison of the four schemes a, b, c, and d, it is found that scheme a obtains the highest comprehensive score, so it is selected as the best weight strategy. The 7 weighting strategies include:

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ERROR COMPARISON OF DP, TD-TR, AND SEB ALGORITHMS WHEN CR = 5

Dataset	Algorithms	PED error (m)	SED error (m)	Speed error (m/s)	Direction error (m)	DTW distance
Berlin_0.05	DP	**0.497**	5.709	1.916	0.015	114.778
	TD-TR	0.865	**2.197**	0.816	0.019	128.447
	SEB	0.609	2.441	**0.806**	**0.014**	**99.433**
Walking	DP	**3.033**	7.111	0.506	0.249	5.097
	TD-TR	3.123	**6.162**	0.479	0.261	5.698
	SEB	3.073	6.539	**0.475**	**0.244**	**5.089**
Airline	DP	**94.430**	1067.689	6.563	**0.008**	4126.591
	TD-TR	133.676	**273.961**	3.564	0.017	2950.760
	SEB	118.867	383.279	**3.503**	0.012	**2648.498**
Illinois	DP	**18.064**	295.895	49.585	0.034	8900.473
	TD-TR	27.349	**57.037**	19.679	0.041	6414.159
	SEB	25.566	60.048	**19.442**	**0.031**	**6165.338**
T-drive	DP	**12.967**	138.147	13.862	0.651	140.033
	TD-TR	22.365	**29.047**	6.466	0.623	19.157
	SEB	19.933	40.779	**3.208**	**0.434**	**18.954**

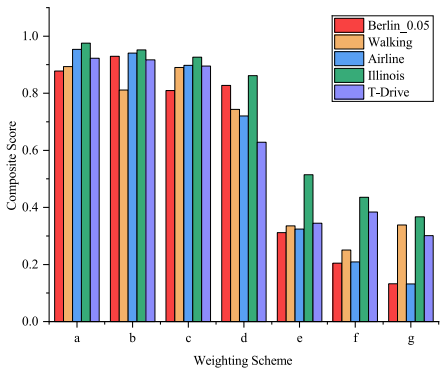


Fig. 9. Changes in the composite scores of the four data sets under different weighting schemes.

- *Scheme a:* W = [0.3, 0.5, 0.1, 0.1], focusing on SED (larger than PED).
- *Scheme b:* W = [0.4, 0.4, 0.1, 0.1], PED and SED are equally important.
- *Scheme c:* W = [0.5, 0.3, 0.1, 0.1], focusing on PED (larger than SED).
- *Scheme d:* W = [0.25, 0.25, 0.25, 0.25], balanced consideration of PED, SED, SAD, and DAD.
- *Scheme e:* W = [0.1, 0.1, 0.3, 0.5], DAD has a higher weight than SAD.
- *Scheme f:* W = [0.1, 0.1, 0.4, 0.4], SAD and DAD are equally important.
- *Scheme g:* W = [0.1, 0.1, 0.5, 0.3], SAD has a higher weight than DAD.

The experiment calculates the comprehensive scores of these 7 weight schemes on 5 data sets. The score obtains by averaging all normalized indicators, and the formula is $Score_{\text{composite}} = \frac{\sum(\text{normalized_index})}{\text{num_index}}$. Among them, normalized_index includes all error indicators that need to be minimized (such as average PED error, maximum PED error, average SED error, maximum SED error, average SAD error, maximum SAD error, average DAD error, maximum DAD error, DTW distance, compression time) and the compression ratio that needs to be maximized. For the error-index that needs to be minimized, the normalization method is $\text{value}_{\text{normalized}} = \frac{\text{value}_{\text{maximum}} - \text{value}_{\text{current}}}{\text{value}_{\text{maximum}} - \text{value}_{\text{minimum}}}$, so that the smaller the original error, the higher the corresponding normalized value (approaching 1); for the index that needs to be maximized (such as compression ratio), $\text{value}_{\text{normalized}} = \frac{\text{value}_{\text{current}} - \text{value}_{\text{minimum}}}{\text{value}_{\text{maximum}} - \text{value}_{\text{minimum}}}$ is used. When the original value is larger, its normalized value is higher (also approaching 1). Based on the above calculation results, we finally confirm that scheme a (W=[0.3, 0.5, 0.1, 0.1]) achieves the highest comprehensive score in multiple indicators, and thus becomes the best weight strategy.

Table III shows the error comparison of DP, TD-TR, and SEB algorithms when CR = 5. Since DP and TD-TR algorithms use PED and SED distances as error metrics, they perform better on PED errors and SED errors, respectively. The SEB algorithm is based on score, and its purpose is to balance the errors of each feature so that the comprehensive loss of the simplified trajectory is as small as possible.

1) For Berlin_0.05, Walking, Illinois, and T-drive datasets, SEB performs slightly worse than the DP on PED error and slightly worse than TD-TR on SED error. However, SEB outperforms the DP and TD-TR in speed error and direction error.
2) For the Airline dataset, the performance of SEB in direction error is inferior to that of DP. This is because the aircraft route often has little change in direction. DP adopts the vertical distance method, which is more advantageous for data with little change in direction.
3) It is worth noting that the DTW distance of SEB on the five datasets is significantly better than other algorithms. The DTW distance is used to evaluate the similarity between the simplified and original trajectories. Compared with a single error metric, it better reflects the comprehensive loss caused by simplified trajectories.

In general, SEB is close to DP and TD-TR in terms of PED error and SED error, respectively; however, it performs better in speed error, direction error, and DTW distance. Therefore, SEB balances the error introduced by compression.

D. Performance of CFPF Algorithm

In this section, we show the performance of the proposed CFPF algorithm compared with the state-of-the-art multiple

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

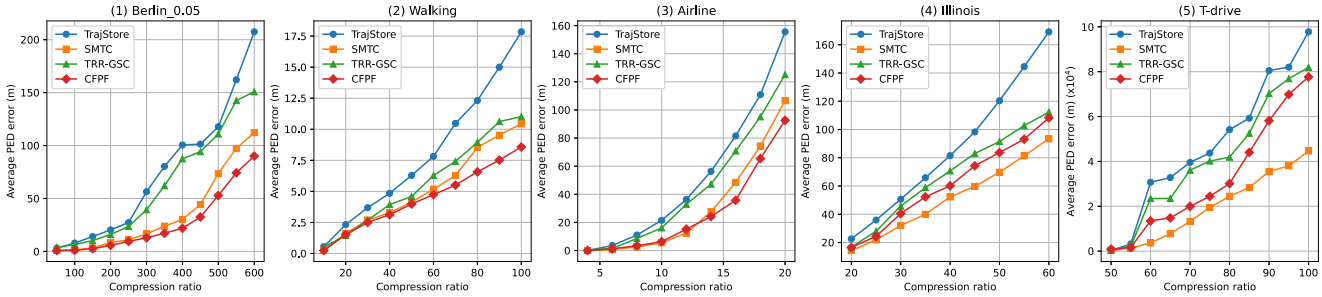


Fig. 10. Compression ratios and PED errors of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

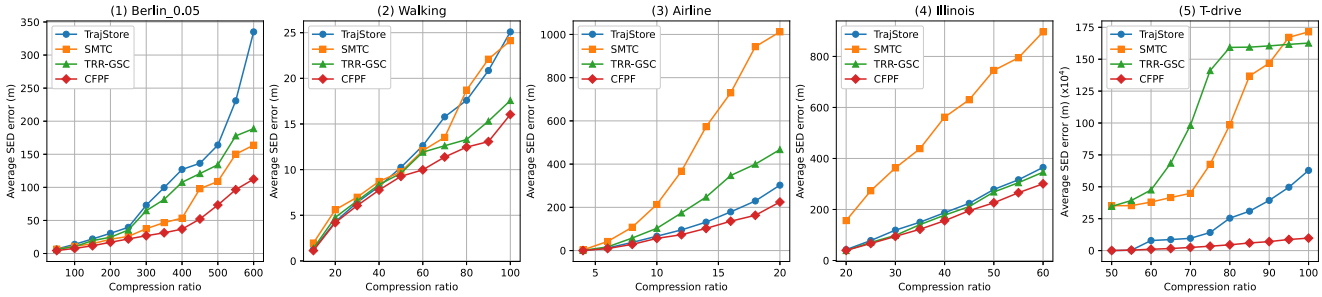


Fig. 11. Compression ratios and SED errors of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

trajectory compression methods in the five datasets. According to the principle, we implemented three representative compression algorithms, namely TrajStore [15], SMTC [16], and TRR-GSC in REST [18]. Considering that the Illinois dataset is large and the execution time of the above algorithms is long, we divide it into 11 experiments and take 20 trajectories for compression each time. The Berlin_0.05, Airline, and Walking datasets are fully compressed. In experiments, SMTC uses DP [8] as the internal single trajectory compression method. In TRR-GSC, the reference set is trained in advance, and the parameters are set as follows: the minimum support and minimum length thresholds are set to 4 and 3 respectively; the distance deviation thresholds are set to 50 m, 10 m, 500 m, 50 m, and 50 m in the five datasets. The parameter μ of the CFPF algorithm is set to 50 m, 10 m, 200 m, 50 m, and 50 m in the five datasets respectively.

1) Error Evaluation: Fig. 10 shows that the compression effect of CFPF on PED error is generally better than TrajStore, SMTC, and TRR-GSC. For the first three datasets, the advantages of CFPF gradually become more significant as the compression ratio increases. In the case of the same PED error, CFPF has at most 25% higher compression ratio than SMTC. For the Illinois dataset, CFPF is slightly worse than SMTC, because the Illinois dataset is collected by two members separately, and there are certain differences in their motion patterns. At the same time, SMTC is equipped with DP algorithm that uses PED distance as the error metric, and its performance in PED error is relatively outstanding. In the T-drive dataset, which consists of trajectories from multiple vehicles, there are greater variations in the motion patterns, leading to a relatively lower performance of CFPF compared to SMTC.

Fig. 11 shows that the SED error of CFPF is significantly lower than TrajStore, SMTC, and TRR-GSC under different compression ratios. For the Berlin_0.05 dataset, we strictly restrict the similarity of inner trajectories, so SMTC performs better than TrajSotre and TRR-GSC. For the real collected datasets Walking, Airline, Illinois, and T-drive, the effect of SMTC is poor. In the case of the same SED error, the compression ratio of CFPF is at most 100% higher than SMTC, 56% higher than TRR-GSC, and 42% higher than TrajStore.

In Fig. 12, TRR-GSC seems to perform poorly on the Walking dataset. When collecting our walking trajectory data, we don't always maintain a certain speed, but stop and go. This has a certain impact on the construction of the reference set in TRR-GSC. Of course, CFPF seems to be affected as well, and it performs worse than TrajStore when CR < 50. Trajstore fully retains each position point in the trajectory and replaces its spatial information with the time mapping value of the reference trajectory. For a trajectory point, replacing it with an adjacent location sometimes results in less feature loss than omitting it. However, as the compression ratio continues to increase, the compression effect of CFPF is better than TrajStore. Under the same speed error, the compression ratio of CFPF is at most 100% higher than that of TrajStore, TRR-GSC, and SMTC.

Fig. 13 illustrates the advantages of CFPF on direction errors. For Berlin_0.05, Walking datasets,and T-drive, under the same direction error, the compression ratio of CFPF is at most 114% higher than TrajStore, 100% higher than TRR-GSC, and 35% higher than SMTC. For Airline and Illinois datasets, SMTC outperforms CFPF when the compression ratio is low. However, as the compression ratio increases, it can be seen that the advantages of CFPF are gradually significant.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

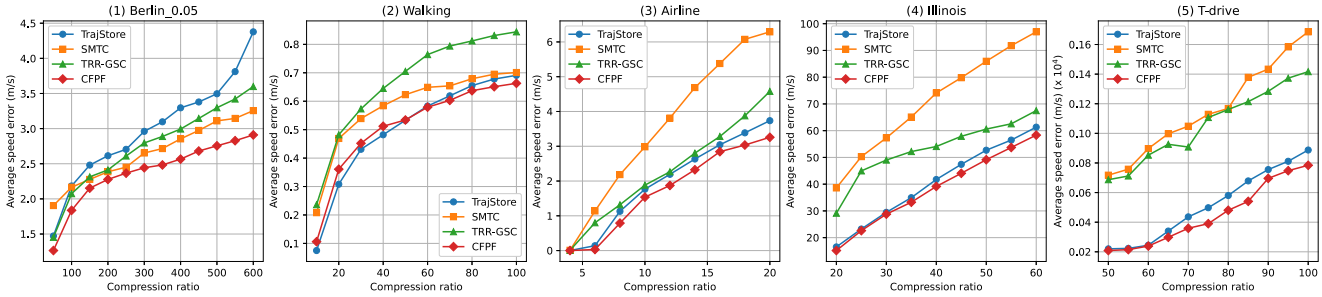


Fig. 12. Compression ratios and speed errors of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

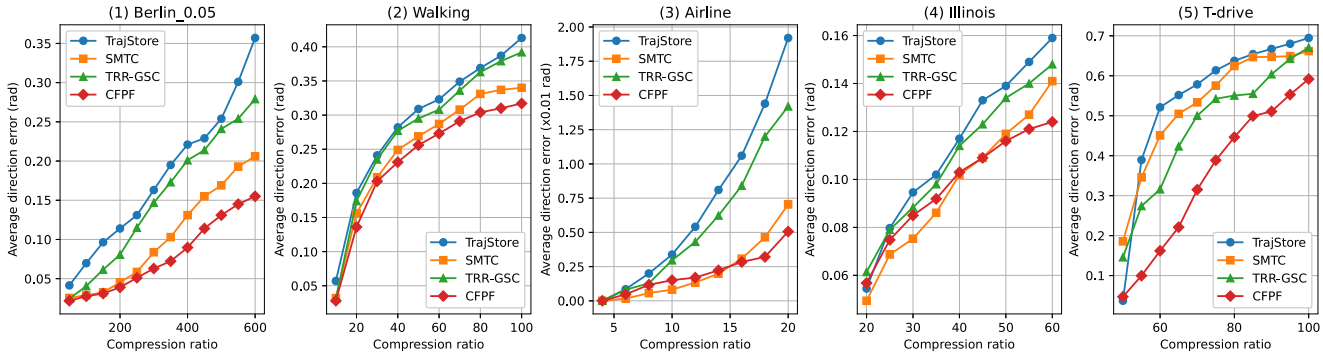


Fig. 13. Compression ratios and direction errors of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

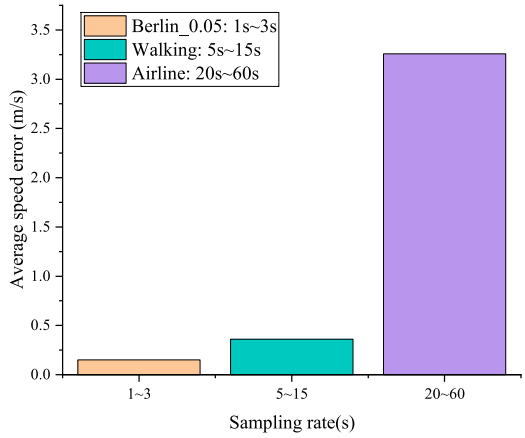


Fig. 14. Average Speed Error at Various Sampling Rates with CR $= 20$.

Fig. 14 shows the change of SAD with different sampling rates when the compression ratio is 20% . We select the three datasets with different sampling rates, Berlin, Walking, and Airline. We find that as the sampling interval increases, the impact of SAD on the simplified results becomes more significant, and the points of speed features can be more effectively retained in low sampling rate scenarios. The reason is that when the sampling interval is larger (that is, the lower the sampling frequency, the larger $t_{i+1} - t_i$), the speed variation range covered by each sampling point is also larger; if the motion behavior is relatively violent (the speed or direction changes are large), the speed difference between adjacent sampling points is more obvious. From formula (5), we can see that the SAD value also increases, thereby increasing the importance of these sampling points in the simplification process.

2) Similarity Evaluation: Fig. 15 illustrates the compression performance of CFPF compared to TrajStore, TRR-GSC, and SMTC on the DTW distance. The smaller the average DTW distance, the higher the similarity between the initial trajectory and the compressed trajectory, which is more convincing than a single PED error, SED error, speed error, and direction error.

1) For the Berlin_0.05 and T-drive datasets, the average DTW distance of CFPF is significantly lower than TrajStore, TRR-GSC, and SMTC under the same compression ratio. Also, the growth rate (slope of the curve) of the DTW distance is higher when the compression ratio is low and slows down as the compression ratio increases.
2) For the Walking dataset, when CR < 40, the average DTW distances of the four algorithms are close. Subsequently, the growth rate of DTW distance corresponding to CFPF slows down significantly, while TrajStore and SMTC still maintained the previous growth rate. When CR $= 100$, the average DTW distance of CFPF is 15% lower than the other three algorithms.
3) For Airline and Illinois datasets, the disadvantages of the SMTC algorithm are clear at a glance. Although the SMTC algorithm directly combines time mapping and trajectory simplification technology, it greatly reduces the similarity between the compressed trajectory and the original trajectory. At this time, the performance of three

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

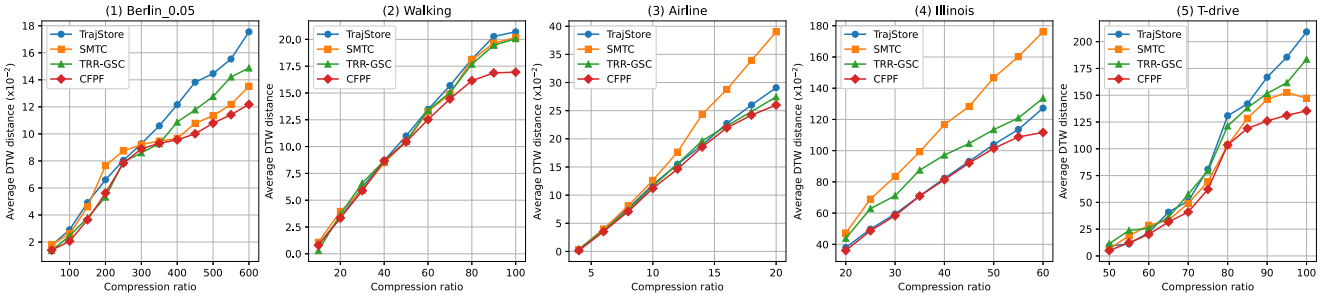


Fig. 15. Compression ratios and DTW distance of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

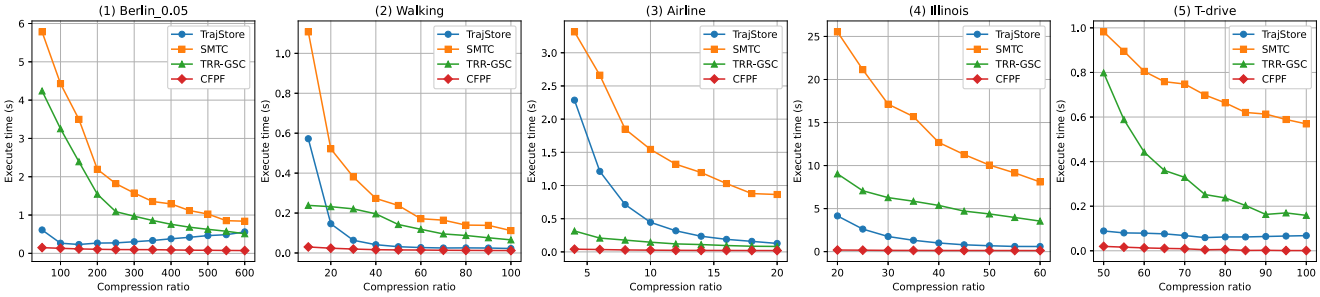


Fig. 16. Time cost of TrajStore, SMTC, TRR-GSC, and CFPF methods on the five datasets.

algorithms CFPF, TRR-GSC and TrajStore is similar. In the Illinois dataset, when CR = 60, the average DTW distance of CFPF is 35% lower than SMTC, 18.5% lower than TRR-GSC, and 12% lower than TrajStore.

3) Time Cost Evaluation: We examine the time overhead of TrajStore, SMTC, TRR-GSC, and CFPF algorithms from two aspects: algorithm execution time and trajectory recovery time. Fig. 16 shows that the execution time of our CFPF algorithm on the five datasets is much lower than the other three methods.

TrajStore clusters each trajectory segment, and then maps the spatial information of the non-reference trajectory to the timestamp of the corresponding position of the reference trajectory. Time mapping needs to traverse the reference trajectory again and again, which brings a certain time overhead. The total time complexity of TrajStore is $O(C_1 \times K^2 \times N^2)$, where C_1 is a constant, K is the number of trajectories, and N is the length of the original trajectory. It is worth noting that in the Berlin_0.05 dataset, the time overhead of TrajStore increases with the increase of the compression ratio after the compression ratio is greater than 150. A reasonable explanation is that as the distance threshold increases, more and more trajectory points are considered time-mappable points by TrajStore, which requires additional traversal of the reference trajectory.

SMTC traverses all compressed trajectories to find the best reference trajectory and time mapping values, and then performs single trajectory compression to select the scheme with the highest compression ratio. This not only brings high time overhead, but also has the problem of trajectory dependence. The total time complexity of SMTC is $O(C_2 \times K^2 \times N^2)$, where C_2 is a relatively large constant because it involves multiple query operations of compressed trajectories. From the five datasets, it can be seen that SMTC has at most 10 times higher time overhead than TrajStore and TRR-GSC, and hundreds of times higher time overhead than CFPF. The time overhead of TRR-GSC is mainly reflected in the sub-trajectory matching of the reference set and the original trajectory. Due to the uncertainty of the set of reference sub-trajectories, it is difficult to accurately calculate the time complexity of TRR-GSC. It can be seen from Fig. 16 that the time complexity of TRR-GSC should be between TrajStore and SMTC, which is also a higher level. Compared to these algorithms, CFPF clusters and fuses a small number of feature points retained after the SEB algorithm, without reciprocating traversal of other trajectories. Therefore, the amount of data is greatly reduced. Therefore, the time overhead of CFPF is extremely low, and it does not have the problem of reference trajectory and trajectory dependence. The total time complexity of CFPF is $O(C \times K \times M \times N \times logM)$, where C is a constant and $M(M \ll N)$ is the length of the simplified trajectory.

In Table V, the trajectory recovery time of CFPF on the five datasets is significantly lower than TrajStore, TRR-GSC, and SMTC. The CFPF algorithm replaces the spatial information of the feature points in the same cluster with the center point. There is no need to find the corresponding spatial information according to the reference trajectory and time mapping value when recovering the trajectory. Therefore, CFPF greatly reduces the trajectory recovery time.

At last, in order to improve the actual user experience, we performed linear fitting on the above experimental data to determine the potential relationship between the compression ratio of the SEB algorithm in the first stage and the compression ratio of CFPF. We have $\mathrm{CR}_2 = -6.2137 + 6.6423 \times \mathrm{CR}_1$, where CR_1

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
QUANTITATIVE AND ERROR METRIC COMPARISON BETWEEN ORIGINAL AND COMPRESSED TRAJECTORIES

Date	Original Points	Retained Points	PED error (m)	SED error (m)	Speed error (m/s)	Direction error (m)	DTW distance
1009	176	66	91.234	224.628	3.201	0.00481	2578.46
1010	205	77	93.567	224.625	3.145	0.00493	2621.59
1011	178	67	92.890	224.624	3.789	0.00499	2540.21
1012	181	68	92.101	224.629	2.998	0.00505	2669.05
1013	183	69	93.210	224.626	3.305	0.00511	2610.74
1014	189	71	91.999	224.622	3.567	0.00519	2598.80
1015	185	70	92.700	224.627	2.954	0.00524	2632.95
1016	192	72	93.123	224.623	3.414	0.00538	2675.21
1017	157	59	92.561	224.622	2.949	0.00587	2476.32

TABLE V
RESTORE TIME OF FOUR METHODS ON THE FIVE DATASETS

	(1) Berlin_0.05	(2) Walking	(3) Airline	(4) Illinois	(5) T-Drive
TrajStore [15]	0.63ms	0.51ms	0.59ms	0.75ms	0.89ms
SMTC [16]	3.08ms	1.55ms	2.16ms	2.59ms	3.62ms
TRR-GSC [18]	0.57ms	0.49ms	0.56ms	1.01ms	0.72ms
CFPF	**0.55ms**	**0.46ms**	**0.52ms**	**0.64ms**	**0.68ms**

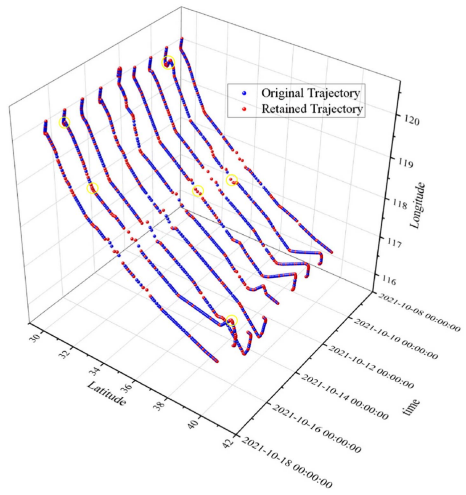


Fig. 17. Comparison of Original and Retained Trajectory Points.

represents the compression ratio of the SEB algorithm in the first stage, which is the inverse of the storage budget ω. CR_2 represents the compression ratio of the CFPF algorithm. The fitting error of the above equation is only 0.466% . For practical use, if the expected compression ratio is 60, that is, $CR_2 = 60$, then the compression ratio CR_1 of the SEB algorithm is best set around 9.97, that is $\omega \approx 0.1$.

E. Case Study

To intuitively demonstrate the effectiveness of the proposed trajectory compression method, we conducted a case study using a representative dataset of real-world aircraft trajectories from the Airline dataset. This dataset was selected due to its clear spatial distribution, significant variations in heading, and the presence of multiple turning points, which make it particularly suitable for evaluating the performance of compression algorithms in preserving essential trajectory features. As illustrated in Fig. 17, the original trajectories (depicted as blue dots) consist of 9 flight paths, each comprising approximately 200 sampling points. After applying the proposed compression algorithm, each trajectory (shown as red dots) is reduced to about 70 points, successfully retaining the main shape and critical turning points, which are highlighted by yellow circles. Table IV summarizes the number of trajectory points before and after compression, as well as key error metrics such as PED, SED, Speed error, Direction error and DTW distance. The results indicate that the proposed method achieves a substantial reduction in data volume while effectively preserving the fundamental characteristics and geometric features of the original trajectories.

VI. CONCLUSION

In this paper, we develop a new spatio-temporal trajectory compression technique, consisting of intra-trajectory error balancing and inter-trajectory feature point clustering. Intra-trajectory error balancing is implemented by the presented SEB algorithm, and inter-trajectory feature point clustering is achieved through proposing CFPF algorithm. SEB balances the error of compressed trajectories by retaining high score trajectory points and provides an efficient solution to the *Min-Error* problem. CFPF reduces the granularity of clustering from trajectory segments to feature points, and extracts the commonality between similar trajectories by clustering and fusing feature point sequences. The combination of SEB and CFPF perfectly achieves intra-trajectory and inter-trajectory redundancy elimination. Extensive experiments based on five real trajectory datasets confirm that our technique performs better than state-of-the-art algorithms in terms of compression ratio, compression error, and time overhead. Our technique is efficient for different types (e.g., urban roads, aircraft routes, and walking) of spatio-temporal trajectory data.

ACKNOWLEDGMENT

The authors would like to express their gratitude to the associate editor and anonymous reviewers for their constructive comments which have helped to improve the quality of the paper.

REFERENCES

[1] Y. Takeuchi and M. Sugimoto, "An outdoor recommendation system based on user location history," in *Proc. Int. Conf. Ubiquitous Intell. Comput.*, 2005, pp. 625–636.
[2] Y.-H. Yu, J. H. Kim, K. Shin, and G.-S. Jo, "Recommendation system using location-based ontology on wireless internet: An example of collective intelligence by using 'mashup' applications," *Expert Syst. Appl.*, vol. 36, pp. 11675–11681, 2009.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.

[3] L. Chen, M. Lv, and G. Chen, "A system for destination and future route prediction based on trajectory mining," *Pervasive Mobile Comput.*, vol. 6, pp. 657–676, 2010.
[4] Z. Feng and Y. Zhu, "A survey on trajectory data mining: Techniques and applications," *IEEE Access*, vol. 4, pp. 2056–2067, 2016.
[5] A. Chen and L. Liu, "Velocity-preserving trajectory compression based on retrace point detection," *Wireless Commun. Mobile Comput.*, vol. 2021, 2021, Art. no. 6674769.
[6] Y. Zhong, "Speed and angle based trajectory simplification algorithm," in *Proc. 2021 Int. Conf. Inf. Technol. Biomed. Eng.*, 2021, pp. 187–192.
[7] D. Amigo, D. S. Pedroche, J. García, and J. M. Molina, "Review and classification of trajectory summarisation algorithms: From compression to segmentation," *Int. J. Distrib. Sensor Netw.*, vol. 17, 2021, Art. no. 15501477211050729.
[8] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica: Int. J. Geographic Inf. Geovisualization*, vol. 10, pp. 112–122, 1973.
[9] N. Meratnia and R. A. de By, "Spatiotemporal compression techniques for moving point objects," in *Proc. Int. Conf. Extending Database Technol.*, 2004, pp. 765–782.
[10] M. Potamias, K. Patroumpas, and T. K. Sellis, "Sampling trajectory streams with spatiotemporal criteria," in *Proc. 18th Int. Conf. Sci. Stat. Database Manage.*, 2006, pp. 275–284.
[11] J. Muckell, J.-H. Hwang, V. Patil, C. T. Lawson, F. Ping, and S. S. Ravi, "Squish: An online approach for GPS trajectory compression," in *Proc. 2nd Int. Conf. Comput. Geospatial Res. Appl.*, 2011, pp. 1–8.
[12] C. Long, R. C. W. Wong, and H. V. Jagadish, "Trajectory simplification: On minimizing the direction-based error," *Proc. VLDB Endowment*, vol. 8, pp. 49–60, 2014.
[13] X. Lin, S. Ma, J. Jiang, Y. Hou, and T. Wo, "Error bounded line simplification algorithms for trajectory compression: An experimental evaluation," *ACM Trans. Database Syst.*, vol. 46, pp. 1–44, 2021.
[14] P. Sun, S. Xia, G. Yuan, and D. Li, "An overview of moving object trajectory compression algorithms," *Math. Problems Eng.*, vol. 2016, pp. 1–13, 2016.
[15] P. Cudré-Mauroux, E. Wu, and S. Madden, "Trajstore: An adaptive storage system for very large trajectory data sets," in *Proc. IEEE 26th Int. Conf. Data Eng.*, 2010, pp. 109–120.
[16] J. Birnbaum, H.-C. Meng, J.-H. Hwang, and C. T. Lawson, "Similarity-based compression of GPS trajectory data," in *Proc. 4th Int. Conf. Comput. Geospatial Res. Appl.*, 2013, pp. 92–95.
[17] Y. Zhao et al., "Rest: A reference-based framework for spatio-temporal trajectory compression," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2797–2806.
[18] K. Zheng, Y. Zhao, D. Lian, B. Zheng, G. Liu, and X. Zhou, "Reference-based framework for spatio-temporal trajectory compression and query processing," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 11, pp. 2227–2240, Nov. 2020.
[19] U. Ramer, "An iterative procedure for the polygonal approximation of plane curves," *Comput. Graph. Image Process.*, vol. 1, pp. 244–256, 1972.
[20] J. Hershberger and J. Snoeyink, *Speeding Up the Douglas-Peucker Line-Simplification Algorithm*. Vancouver, Canada: Univ. British Columbia, 1992.
[21] T. Li, R. Huang, L. Chen, C. S. Jensen, and T.B. Pedersen, "Compression of uncertain trajectories in road networks," *Proc. VLDB Endowment*, vol. 13, no. 7, pp. 1050–1063, 2020.
[22] M. Laass, M. Kiermeier, and M. Werner, "Improving persistence based trajectory simplification," in *Proc. 22nd IEEE Int. Conf. Mobile Data Manage.*, 2021, pp. 157–162.
[23] H. Li et al., "Unsupervised hierarchical methodology of maritime traffic pattern extraction for knowledge discovery," *Transp. Res. Part C: Emerg. Technol.*, vol. 143, 2022, Art. no. 103856.
[24] X. Bai, Z. Xie, X. Xu, and Y. Xiao, "An adaptive threshold fast DBSCAN algorithm with preserved trajectory feature points for vessel trajectory clustering," *Ocean Eng.*, vol. 280, 2023, Art. no. 114930.
[25] Y. Xu et al., "Improved vessel trajectory prediction model based on stacked-bigrus," *Secur. Commun. Netw.*, vol. 2022, no. 1, 2022, Art. no. 8696558.
[26] G. Trajcevski, H. Cao, P. Scheuermann, O. Wolfson, and D. Vaccaro, "On-line data reduction and the quality of history in moving objects databases," in *Proc. 5th ACM Int. Workshop Data Eng. Wireless Mobile Access*, 2006, pp. 19–26.
[27] J. Liu, K. Zhao, P. Sommer, S. Shang, B. Kusy, and R. Jurdak, "Bounded quadrant system: Error-bounded trajectory compression on the go," in *Proc. IEEE 31st Int. Conf. Data Eng.*, 2015, pp. 987–998.
[28] J. Liu et al., "A novel framework for online amnesic trajectory compression in resource-constrained environments," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 11, pp. 2827–2841, Nov. 2016.
[29] Z. Liu et al., "An online method for ship trajectory compression using ais data," *J. Navigation*, vol. 77, pp. 1–22, 2024.
[30] Y. Li and H. Ren, "Visual analysis of vessel behaviour based on trajectory data: A case study of the yangtze river estuary," *ISPRS Int. J. Geo- Inf.*, vol. 11, no. 4, 2022, Art. no. 244.
[31] R. Yan, H. Mo, D. Yang, and S. Wang, "Development of denoising and compression algorithms for ais-based vessel trajectories," *Ocean Eng.*, vol. 252, 2022, Art. no. 111207.
[32] T. Li, L. Chen, C. S. Jensen, and T. B. Pedersen, "TRACE: Real-time compression of streaming trajectories in road networks," *Proc. VLDB Endowment*, vol. 14, no. 7, pp. 1175–1187, 2021.
[33] C. Long, R. C.-W. Wong, and H. V. Jagadish, "Direction-preserving trajectory simplification," in *Proc. VLDB Endow.*, vol. 6, pp. 949–960, 2013.
[34] A. Nibali and Z. He, "Trajic: An effective compression system for trajectory data," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 11, pp. 3138–3151, Nov. 2015.
[35] J. Tao, L. Chen, and J. Fang, "A novel real-time trajectory compression method for privacy protection," in *Proc. IEEE 9th Int. Conf. Data Sci. Adv. Anal.*, 2022, pp. 1–10.
[36] D. Zhang, M. Ding, D. Yang, Y. Liu, J. Fan, and H. T. Shen, "Trajectory simplification: An experimental study and quality analysis," *Proc. VLDB Endowment*, vol. 11, pp. 934–946, 2018.
[37] J. Mao, Q. Song, C. Jin, Z. Zhang, and A. Zhou, "Online clustering of streaming trajectories," *Front. Comput. Sci.*, vol. 12, pp. 245–263, 2018.
[38] D. J. Berndt and J. Clifford, "Using dynamic time warping to find patterns in time series," in *Proc. 3rd Int. Conf. Knowl. Discov. Data Mining*, 1994, pp. 359–370.



Lei Yang (Member, IEEE) received the PhD degree in computer science from the College of Information Science and Engineering, Hunan University, Changsha, China, in 2013. He is currently a professor with the College of Information Science and Engineering, Hunan University, China. His current research interests include distributed computing and storage, edge computing in the Industrial Internet, Big Data processing, and machine learning.



Xin Cheng received the MS degree in software engineering from the North University of China, Taiyuan, in 2021. She is currently working toward the PhD degree in the College of Computer Science and Electronic Engineering, Hunan University. Her research interests focus on data compression, machine learning, and edge intelligence.



Yuwei Liao received the MS degree in computer technology from Hunan University, Changsha, China, in 2023. His research interests were mainly in Big Data management and edge computing. He is currently working at Huawei Technologies Co., Ltd.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.



Rui Li (Member, IEEE) received the PhD degree in computer science from Hunan University, China. He is currently an associate professor of computer science and engineering with the same institution. He was a visiting scholar with Nagoya University, Japan, from 2012 to 2013. His research interests include intelligent medical and healthcare, machine learning, embedded and real-time systems.



Guoqi Xie (Senior Member, IEEE) received the PhD degree in computer science and engineering from Hunan University, China, in 2014. He is currently a professor with Hunan University. He was a postdoctoral research fellow with Nagoya University, Japan. His current research interests include embedded systems, real-time systems, and automotive electronics. He received the 2018 IEEE TCSC Early Career Researcher Award. He is currently serving on the editorial boards of the Journal of Systems Architecture, the Microprocessors and Microsystems, and the Journal of Circuits, Systems, and Computers. He is an ACM senior member.

Authorized licensed use limited to: Fuzhou University. Downloaded on April 16,2026 at 11:37:12 UTC from IEEE Xplore. Restrictions apply.